M504 AI and Applications

Dr. Sanjit Kumar Saha

Gisma
University
of Applied
Sciences

About Me

- PhD (Dr.-Ing.) in Computer Science
- Expertise: Machine Learning, Data Mining, Artificial Intelligence (AI)
- 15+ years teaching & research experience
- Experience in industry-oriented AI projects
- Tools: Python, TensorFlow, PyTorch
- Focus: Practical & applied learning

M504 AI and Applications

Objectives

Specific objectives include

- To evaluate core concepts of Python programming for data science.
- To build data science pipelines with Python in action.

Learning Outcomes

On successful completion of the module, students will be able to

- Critically evaluate the business contexts that can benefit from Python programming.
- Critically analyze core Python programming concepts, including its core structures and data science libraries.
- Critically build data.

Looking Forward

- Interactive theoretical and practical sessions
- Hands-on coding and real-world examples
- Open discussions and questions

Assessment Overview

- 70% Individual Project
- 15% Online Assessments
- 15% Class Participation
- Deadline: September 18, 2026 at 18:00 Berlin Time

Your Role

- Act as a Data Science Consultant
- Analyze a real-world dataset
- Answer business questions
- Present actionable insights

Submission Requirements

- Submit a Jupyter Notebook as HTML
- Upload on Canvas
- Notebook size below 20,000 characters
- Accept Declaration of Authorship

Project Structure

- Business Context
- Data Exploration
- Data Preprocessing
- Explanatory Data Analysis
- Final Discussion and Conclusion

Business Context

- Who is the client company?
- What type of data is available?
- Why is analysis important?
- Define the business problem clearly

Data Exploration

- Understand dataset characteristics
- Check missing values
- Identify duplicates and anomalies
- Explore patterns and metadata

Data Preprocessing

- Handle missing values
- Convert data formats
- Create new features
- Prepare data for analysis

Explanatory Data Analysis

- Define 5–10 meaningful business questions
- Use Python and Pandas
- Explain business importance
- Interpret results clearly

Example Business Questions

- Best month for sales?
- Top-performing products?
- Best time for advertising?
- Frequently bought products together?

For Each Question

- Add subsection in notebook
- Explain importance
- Write runnable Python code
- Interpret findings and recommendations

Final Discussion & Conclusion

- Summarize key insights
- Discuss strengths and limitations
- Provide business recommendations
- Connect analysis to decision-making

Academic Integrity

- Work must be your own
- Avoid plagiarism
- Understand every line of code
- Follow Harvard referencing

Marking Criteria

- 35% Code correctness and completeness
- 35% Report structure and writing quality
- Critical evaluation of results
- Professional presentation

Tips for High Marks

- Choose meaningful datasets
- Use good visualizations
- Write clear explanations
- Think like a consultant

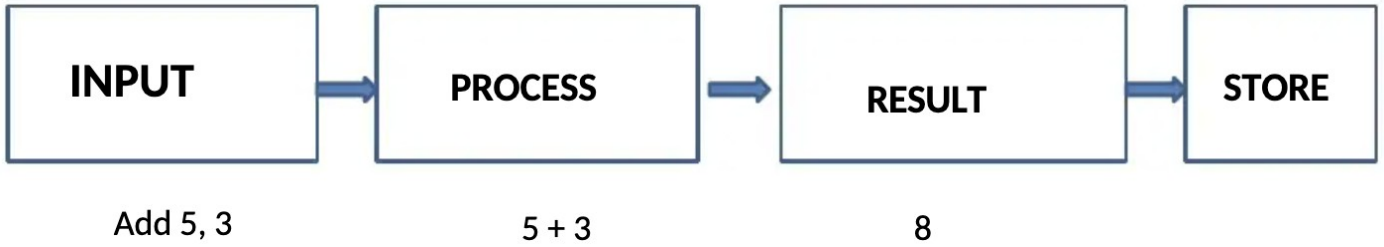
Closing

- Start early and plan carefully
- Ask questions whenever needed
- Focus on both analysis and communication

Introduction to Computer Programming with Python

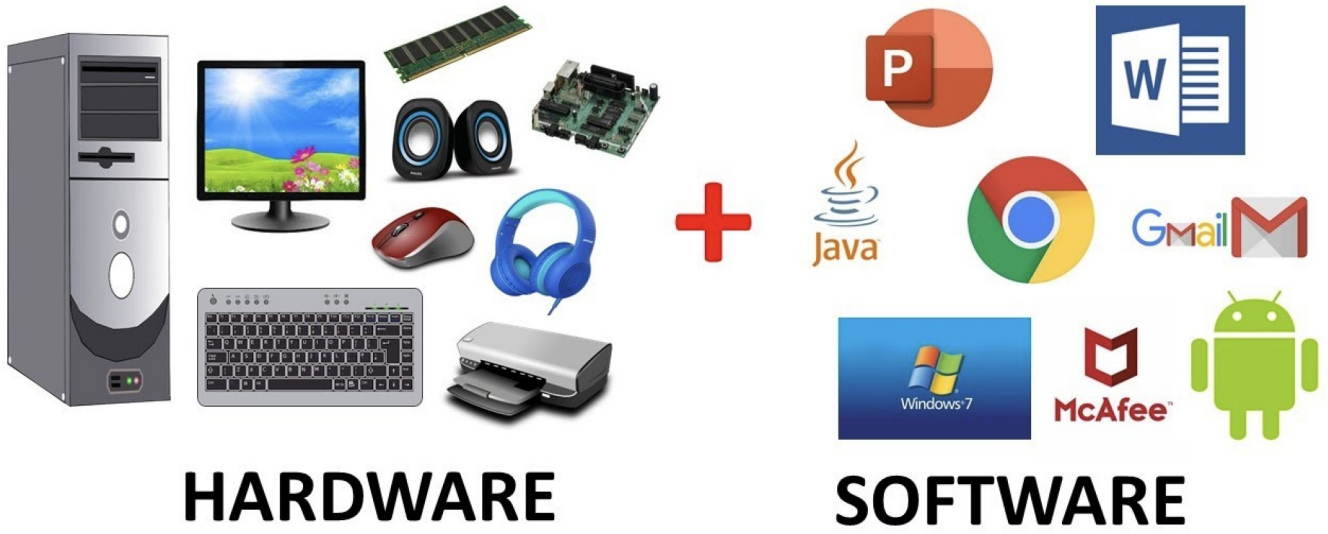
Computer

- Electronic device
- Takes input (instructions from the user), process it and gives the desired output (result)



Components

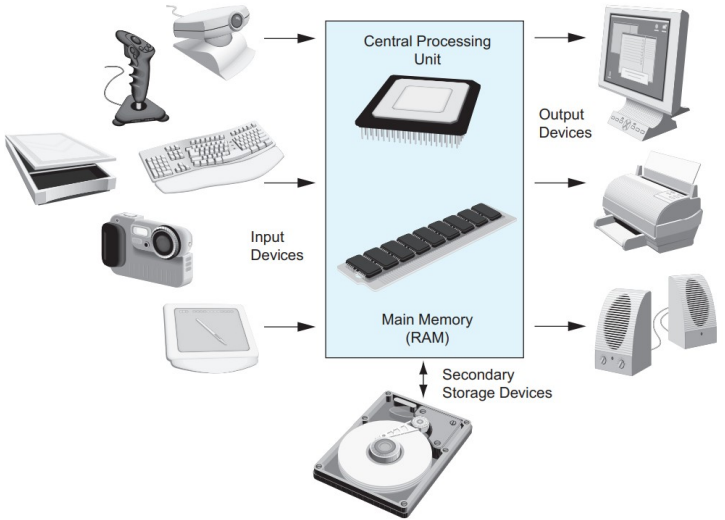
- ***Hardware***
  - mounted physical components
- ***Software***
  - computer program that directs the computer hardware



Hardware

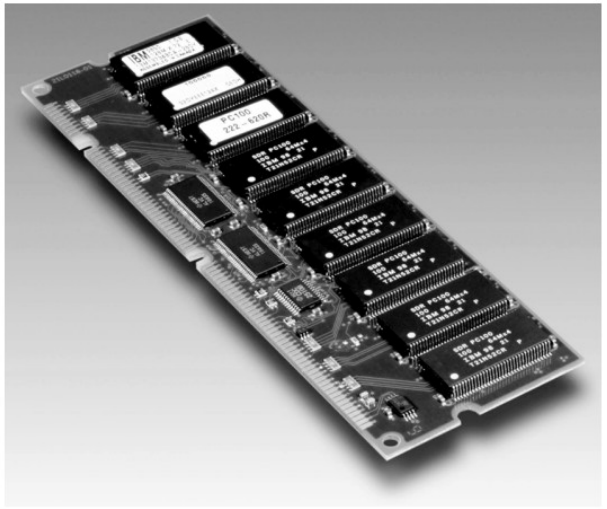
https://www.pearsonhighered.com/assets/samplechapter/0/3/2/1/0321537114.pdf

- Hardware refers to all of the physical devices, or components, that a computer is made of
  - A computer is not one single device, but a system of devices that all work together
  - Each device in a computer plays its own part
    - Like the different instruments in a symphony orchestra
- A typical computer system consists of the following major components
  - The central processing unit (CPU)
  - Main memory
  - Secondary storage devices
  - Input devices
  - Output devices



Main Memory

- You can think of main memory as the computer's work area
  - This is where the computer stores
    - A program while the program is running
    - The data that the program is working with
- Main memory is commonly known as random-access memory, or RAM
  - It is called this because the CPU is able to quickly access data
    - Stored at any random location in RAM
  - RAM is usually a volatile type of memory
    - That is used only for temporary storage while a program is running
    - When the computer is turned off, the contents of RAM are erased
  - Inside your computer, RAM is stored in chips



How a Program Works

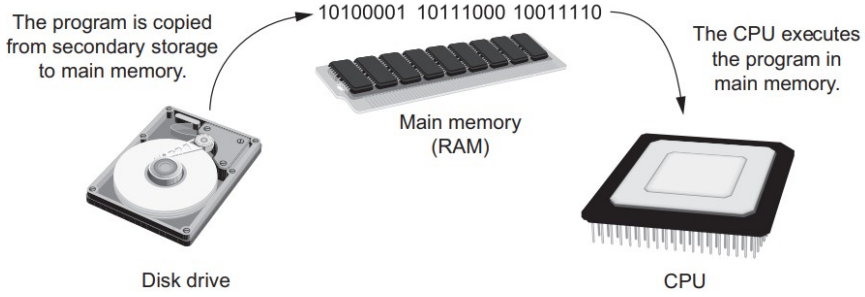
https://www.pearsonhighered.com/assets/samplechapter/0/3/2/1/0321537114.pdf

- A program is nothing more than a list of instructions
  - That cause the CPU to perform operations
  - Each instruction in a program is a command that tells the CPU to perform a specific operation
    - The CPU does nothing on its own
      - It has to be told what to do
        - That is the purpose of a program

How a Program Works

https://www.pearsonhighered.com/assets/samplechapter/0/3/2/1/0321537114.pdf

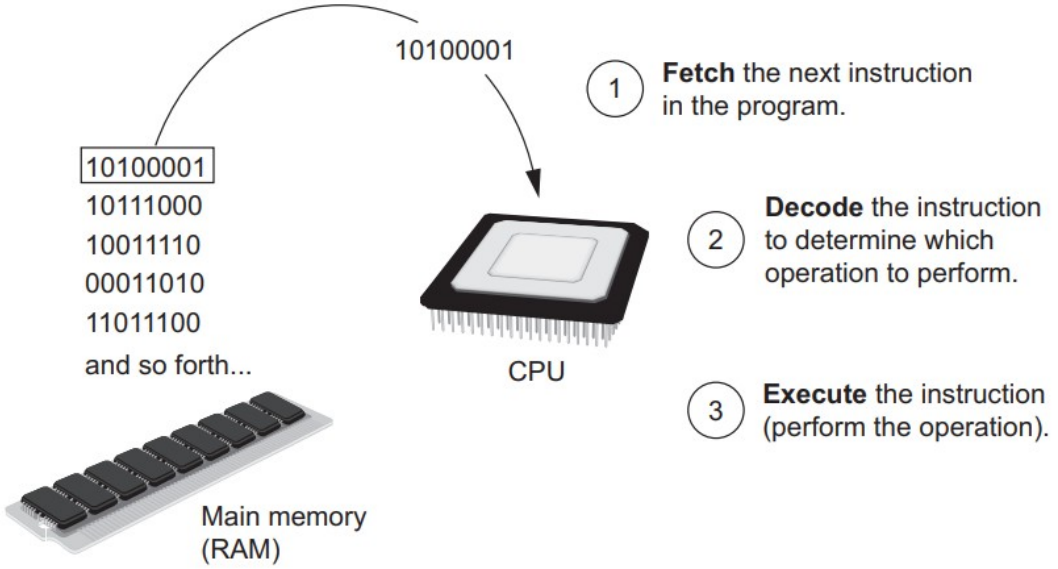
- Programs are usually stored on a secondary storage device
  - Such as a disk drive
- The program has to be copied into main memory, or RAM, each time the CPU executes it
  - For example, suppose you have a word processing program on your computer's disk
    - To execute the program you use the mouse to double-click the program's icon
    - This causes the program to be copied from the disk into main memory
    - Then, the computer's CPU executes the copy of the program that is in main memory



How a Program Works

https://www.pearsonhighered.com/assets/samplechapter/0/3/2/1/0321537114.pdf

- The CPU is engaged in a process that is known as the fetch-decode-execute cycle
  - When it executes the instructions in a program
- This cycle is repeated for each instruction in the program
  - Fetch
  - Decode
  - Execute

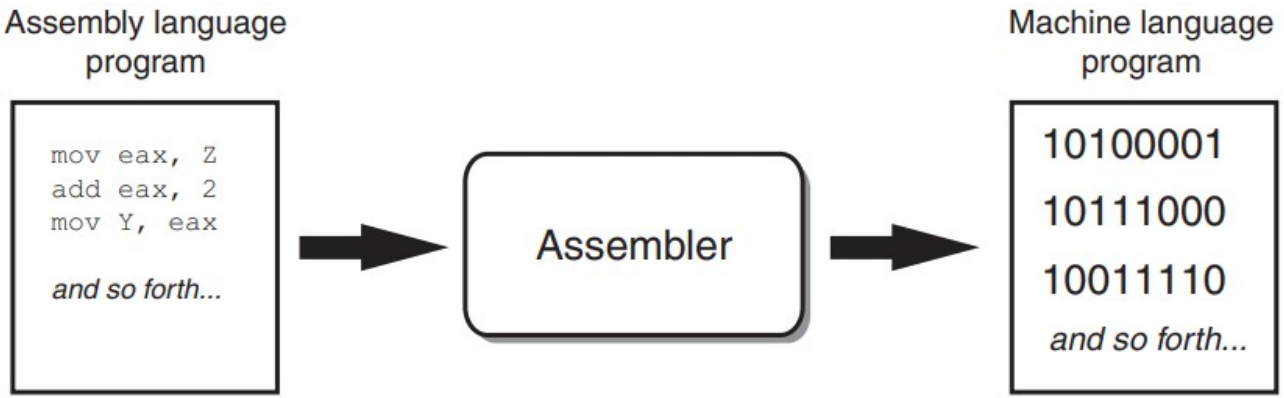


From Machine Language to Assembly Language

- Computers can only execute programs that are written in machine language
  - But it is impractical for people to write programs in machine language
- Programming in machine language would also be very difficult
  - A program can have thousands or even millions of binary instructions
    - Writing such a program would be very tedious and time consuming
  - Putting a 0 or a 1 in the wrong place will cause an error

From Machine Language to Assembly Language

- For this reason, assembly language was created
  - Instead of using binary numbers for instructions, assembly language uses short words
    - That are known as mnemonics
- Assembly language programs cannot be executed by the CPU, however
  - The CPU only understands machine language
  - So, a special program known as an assembler is used
    - To translate an assembly language program to a machine language program



High-Level Languages

https://www.pearsonhighered.com/assets/samplechapter/0/3/2/1/0321537114.pdf

- In the 1950s, high-level languages began to appear
  - As a new generation of programming languages
  - A high-level language allows you to create powerful and complex programs
    - Without knowing how the CPU works
    - Without writing large numbers of low-level instructions
  - In addition, most high-level languages use words that are easy to understand
- DISPLAY "Hello world"
  - For example, this is an instruction to display the message “Hello world” on the computer screen in COBOL
    - Which was one of the early high-level languages created in the 1950s

High-Level Languages

https://www.pearsonhighered.com/assets/samplechapter/0/3/2/1/0321537114.pdf

- Thousands of high-level languages have been created
  - Since the 1950s

Language	Description
Ada	Ada was created in the 1970s, primarily for applications used by the U.S. Department of Defense. The language is named in honor of Countess Ada Lovelace, an influential and historic figure in the field of computing.
BASIC	Beginners All-purpose Symbolic Instruction Code is a general-purpose language that was originally designed in the early 1960s to be simple enough for beginners to learn. Today, there are many different versions of BASIC.
FORTRAN	FORmula TRANslator was the first high-level programming language. It was designed in the 1950s for performing complex mathematical calculations.
COBOL	Common Business-Oriented Language was created in the 1950s, and was designed for business applications.
Pascal	Pascal was created in 1970, and was originally designed for teaching programming. The language was named in honor of the mathematician, physicist, and philosopher Blaise Pascal.
C and C++	C and C++ (pronounced "c plus plus") are powerful, general-purpose languages developed at Bell Laboratories. The C language was created in 1972 and the C++ language was created in 1983.
C#	Pronounced "c sharp." This language was created by Microsoft around the year 2000 for developing applications based on the Microsoft .NET platform.
Java	Java was created by Sun Microsystems in the early 1990s. It can be used to develop programs that run on a single computer or over the Internet from a web server.
JavaScript	JavaScript, created in the 1990s, can be used in web pages. Despite its name, JavaScript is not related to Java.
Python	Python, the language we use in this book, is a general-purpose language created in the early 1990s. It has become popular in business and academic applications.
Ruby	Ruby is a general-purpose language that was created in the 1990s. It is increasingly becoming a popular language for programs that run on web servers.
Visual Basic	Visual Basic (commonly known as VB) is a Microsoft programming language and software development environment that allows programmers to create Windows-based applications quickly. VB was originally created in the early 1990s.

Key Words, Operators, and Syntax: An Overview

- Each high-level language has its own set of predefined words that the programmer must use to write a program
  - The words that make up a high-level programming language are known as keywords or reserved words
  - Each keyword has a specific meaning and cannot be used for any other purpose
  - The table shows all of the Python keywords

and	del	from	not	while
as	elif	global	or	with
assert	else	if	pass	yiel
break	except	import	print	
class	exec	in	raise	
continue	finally	is	return	
def	for	lambda	try	

Keywords, Operators, and Syntax: An Overview

- Programming languages have operators that perform various operations on data
  - In addition to keywords
  - For example, all programming languages have math operators that perform arithmetic
  - In Python, as well as most other languages, the + sign is an operator that adds two numbers
    - 12 + 75
- Each language also has its own syntax
  - Which is a set of rules that must be strictly followed when writing a program
  - The syntax rules dictate how keywords, operators, and various punctuation characters must be used in a program
  - When you are learning a programming language, you must learn the syntax rules for that particular language

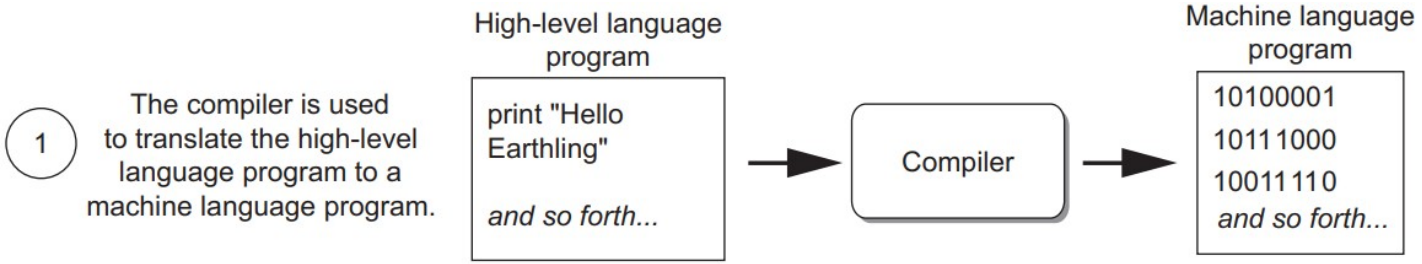
Compilers and Interpreters

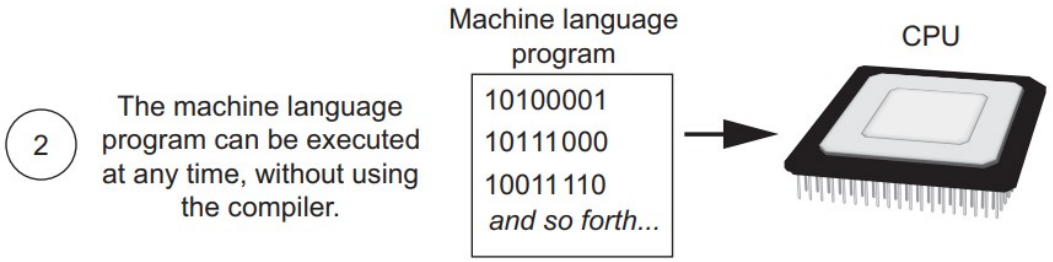
https://www.pearsonhighered.com/assets/samplechapter/0/3/2/1/0321537114.pdf

- Programs that are written in a high-level language must be translated into machine language
  - Because the CPU understands only machine language instructions
- The programmer will use either a compiler or an interpreter to make the translation
  - Depending on the language that a program has been written in

Compilers and Interpreters

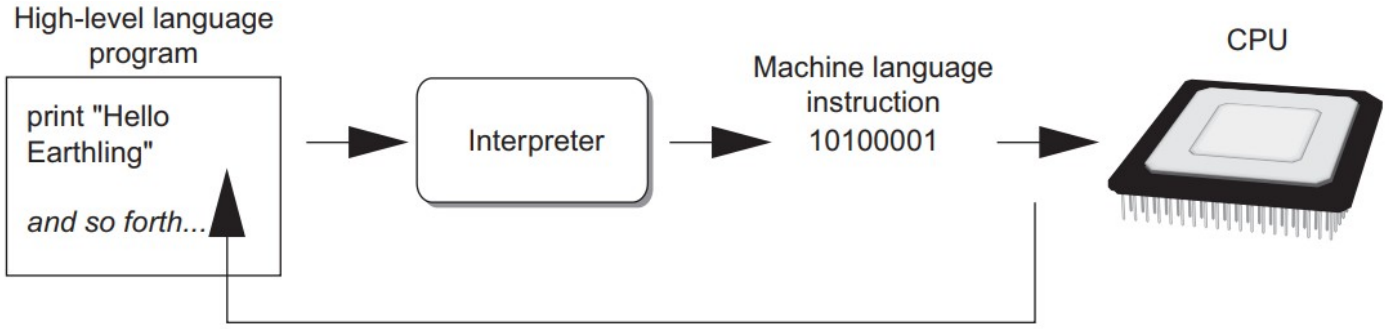
- A compiler is a program that translates a high-level language program into a separate machine language program
  - The machine language program can then be executed any time it is needed
  - As shown in the figure, compiling and executing are two different processes





Compilers and Interpreters

- The Python language uses an interpreter
  - Which is a program that both translates and executes the instructions in a high-level language program
  - The interpreter repeats the following process for every instruction in the program
    - It reads each individual instruction in the program
    - It converts it to machine language instructions
    - It immediately executes them
  - Because interpreters combine translation and execution, they typically do not create separate machine language programs



The interpreter translates each high-level instruction to its equivalent machine language instructions and immediately executes them.

This process is repeated for each high-level instruction.

What is Python?

[Hunt 2019]

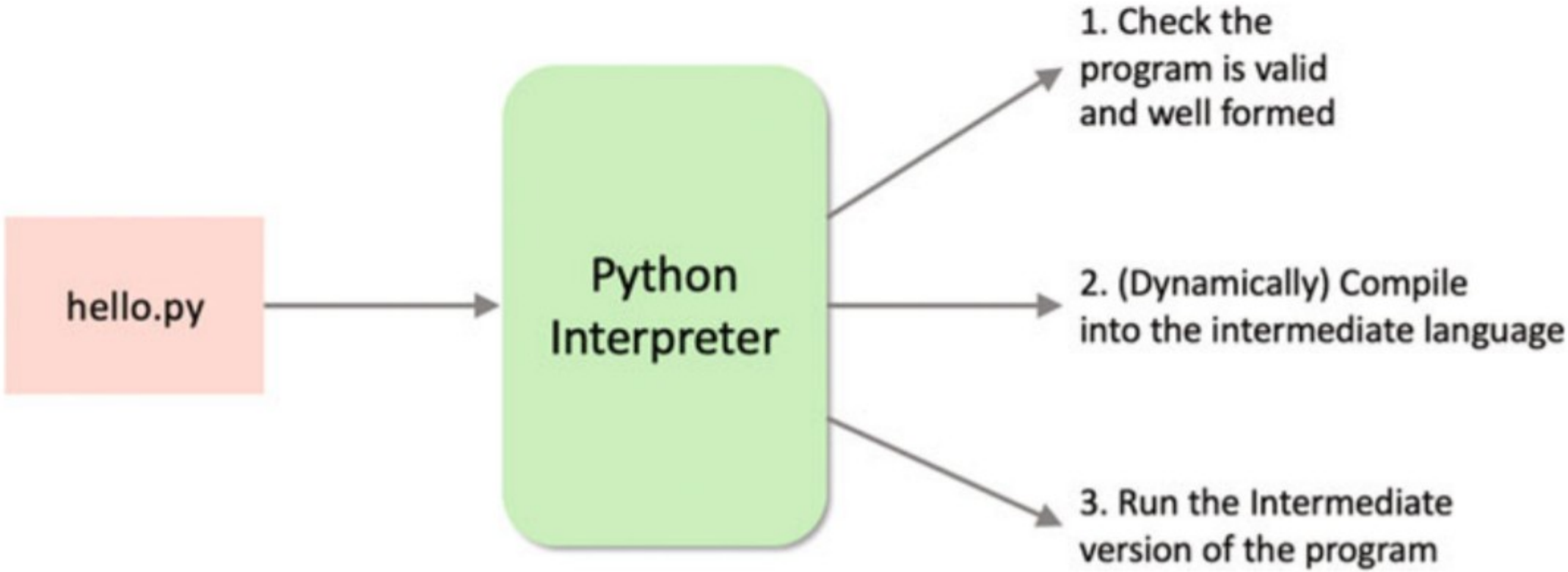
- Python is a general-purpose programming language
  - Originally conceived back in the 1980s by Guido van Rossum at Centrum Wiskunde & Informatica (CWI) in the Netherlands
- Python is now managed by the not-for-profit Python Software Foundation
  - Which was launched in March 2001
  - https://en.wikipedia.org/wiki/Python_Software_Foundation
  - It is also responsible for various processes within the Python community
    - Developing the core Python distribution
    - Managing intellectual rights
    - Supporting developer conferences including PyCon

Why Python?

- Easy to learn and powerful to use
  - Simplest, clean and readable syntax
- Rich ecosystem of libraries
  - NumPy, Scikit-learn, Pandas…
- Strong support for data analysis
  - Clean messy data, Dataset preprocessing, statistical analysis, handle large datasets
- Essential for machine learning and AI
  - Predictive modeling, classification and clustering, Deep learning and neural networks
- Large community and industry demand
  - Tutorial and courses, Continuous updates, Troubleshooting through forums like Stack Overflow
- Integration and Flexibility
  - Databases (SQL, NoSQL), Big data tools (Hadoop), Web frameworks (Django)

Simplified Steps of Interpreter Process

[Hunt 2019]



PyPI and pip

https://en.wikipedia.org/wiki/Python_Package_Index

- The Python Package Index (PyPI)
  - Is a repository of software for the Python programming language
- pip
  - Is a package manager that uses PyPI as the default source for installing packages and their dependencies

Running the First Program on Google Colab

- print()
  - is a predefined function that prints things out to the output stream
  - This output stream of data can be sent to an output window such as the terminal on a Mac or Command Window on a Windows PC
    - In this case, we are printing the string 'Hello World'
- Google Colab
  - Colaboratory, or "Colab" for short, allows you to write and execute Python in your browser, with
    - Zero configuration required
    - Free access to GPUs
    - Easy sharing
  - https://colab.research.google.com/
  - https://github.com/sanjitcse/M504/blob/main/basic_python_exercises.ipynb

-

```
print("Hello world!")
```

Expressions

Expressions

[Adhikari 2017]

- Programs are made up of expressions
  - Which describe to the computer how to combine pieces of data
  - Expressions, such as 3 * 4, are evaluated by the computer
- The grammar rules of a programming language are rigid
  - The computer will show a SyntaxError error if an expression differs from its prescribed expression structures
  - The *Syntax* of a language is its set of grammar rules
- Small changes to an expression can change its meaning entirely
  - 3 * 4 is not equal to 3 ** 4
  - It is a semantic error

```
In [6]: 3 *
  File "/tmp/ipykernel_4622/2069723010.py", line 1
    3 *
      ^
SyntaxError: invalid syntax
```

Operators

https://www.w3schools.com/python/python_operators.asp

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

Arithmetic Operators

[Adhikari 2017]

- Python expressions obey the same familiar rules of precedence as in algebra
  - Parentheses can be used to group together smaller expressions within a larger expression

Expression Type	Operator	Example	Value
Addition	+	*2 + 3*	*5*
Subtraction	-	*2 - 3*	*-1*
Multiplication	*	*2 * 3*	*6*
Division	/	*7 / 3*	*2.66667*
Integer Division	//	*7 // 3*	*2*
Remainder	%	*7 % 3*	*1*
Exponentiation	**	*2 ** 0.5*	*1.41421*

Assignment Operators

[Hunt 2019]

https://www.w3schools.com/python/python_operators.asp

- Compound operators combine together
  - A numeric operation (such as add)
  - With the assignment operator

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Names (i.e., Variables)

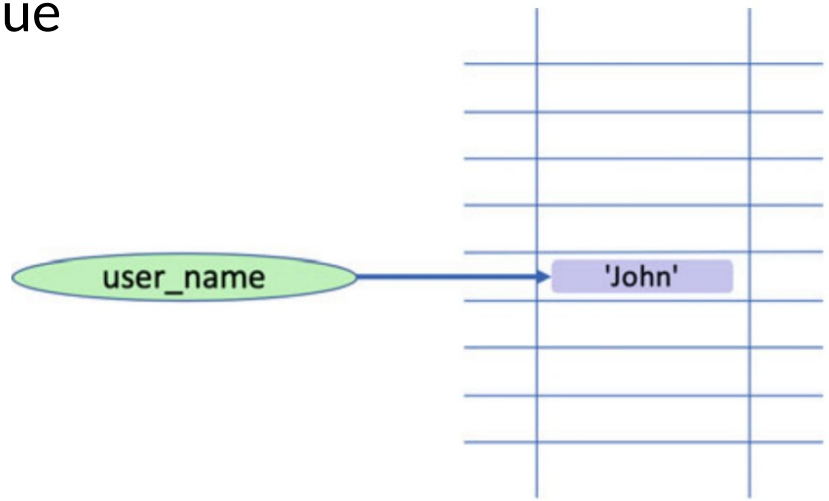
[Adhikari 2017]

- Names are given to values in Python using an assignment statement
  - The value of the expression to the right of = is assigned to the name
    - *a = 3 * 4*
  - A previously assigned name can be used in another expression
    - *b = 2 * a*
- Names must start with a letter, but can contain both letters and numbers
  - A name cannot contain a space
    - It is common to use an underscore character _ to replace each space
  - It is up to the programmer to choose names that are easy to interpret
    - Typically, more meaningful names can be invented than *a* and *b*

Names (i.e., Variables)

[Hunt 2019]

- The names are actually references to a particular space in memory
  - user_name = “John”
    - The name user_name is acting as a label for an area of memory which holds the value “John”
- A name is also called a variable
  - Because the value it references in memory can vary during the lifetime of the program
  - If we want to, we can reuse the variable to hold another value
    - user_name = “Alice”
  - A variable is a named area of the computers’ memory
    - It can hold data



Call Expressions

[Adhikari 2017]

- Call expressions invoke functions, which are named operations
  - The name of the function appears first, followed by expressions in parentheses
  - *abs(-12)*
  - *max(2, 2 + 3, 4)*
    - The max function is called on three arguments
    - The value of each expression within parentheses is passed to the function
    - The function returns the final value of the full call expression
    - The max function can take any number of arguments and returns the maximum
- Operators and call expressions can be used together in an expression
  - *x = 2 + abs(-12)*

Built-In Functions

[Adhikari 2017]

- A few functions are available by default
  - The built-in functions are listed here https://docs.python.org/3/library/functions.html
- The list of built-in functions of Python includes many functions
  - That are never needed in data science applications
    - We will learn and work with the most important functions in context

Expressions vs Statements

[Hunt 2019]

https://fsharpforfunandprofit.com/posts/expressions-vs-statements

- Expression is essentially a computation that generates a value
  - It can be a combination of values and functions
    - *3 * 4 + abs(-12)*
- Statement is just a standalone unit of execution and does not return anything
  - It can be a single or multiple lines of code
    - *print("Hello World!")*

Comments

[Hunt 2019]

- Comments are sections of a program that are ignored by the Python interpreter
  - They are not executable code
  - It is common practice to add comments to code
    - It helps anyone reading the code to understand what the code does
- A comment is indicated by the # character in Python
  - Anything following that character to the end of the line will be ignored by the interpreter

```
# Below, we multiply 3 by 4
x = 3 * 4   # This is also another comment
```